Distributed Online Dispatch for Microgrids Using Hierarchical Reinforcement Learning Embedded With Operation Knowledge

Tianguang Lu, *Member, IEEE*, Ran Hao, *Student Member, IEEE*, Qian Ai, *Senior Member, IEEE*, and Hongying He, *Member, IEEE*

***Abstract*—This paper considers the problem of distributed online economic dispatch (DOED) from sequential data using reinforcement learning. Learning operation behavior in high-dimension environments with constraints is a major challenge for the DOED of networked microgrids (MGs), where insufficient exploration prevents agents from building complex policies. Therefore, this paper develops a hierarchical reinforcement learning (HRL) to handle the DOED problem, where radial basis function (RBF) approximation is incorporated to make policies in continuous space. Based on the hierarchical framework, the HRL algorithm improves learning efficiency and reduces computational cost. The online HRL achieves distributed self-adaption and better performance of real-time dispatch by a modest number of interacting variables. In addition, guided by domain knowledge, the HRL algorithm avoids baseline violation and additional learning beyond feasible action space. In the case of an actual networked MG cluster in Qingdao with real operation data, simulation is conducted to verify that the proposed hierarchical learning can reduce long-term operation costs and enhance operation stability. To explore the learning process, we also provide its convergence condition and analyze the sensitivity of the learning parameters.**

***Index Terms*—Data-driven model, hierarchical reinforcement learning, knowledge guiding, distributed online economic dispatch.**

I. Introduction

THE energy transition from a mode that seriously relies on fossil fuels to one mixed with considerable renewable energy resources (RES) has been expediting recently, bolstered by mounting concerns about climate change and decreasing costs of solar or wind energy [1]. However, high-proportion renewables put forward a higher request for rolling correction and self-adaption in the face of RES prediction biases in the system.

Microgrid (MG) provides a universal interface for different kinds of distributed resources, such as RES, elastic load, and battery energy storage systems (BESS). However, coordinating multiple MGs introduces increasing risks of operation failure and a lower stability margin. In addition, inevitable prediction deviations in various types of renewable energy and demand profiles lead to a large gap between day-ahead plans and actual operation. Therefore, the problem potentially drives a necessary paradigm shift in developing distributed online economic dispatch (DOED) for networked MGs to improve scheduling accuracy further [2].

Prompted by the advance of 5 G and Internet of things technology, the consensus regarding the adoption of real-time sensing and online decision making to support the stable, economic operation is increasing. However, general DOED methods are myopic and can only obtain local optimal solutions. The main challenge is the conflict between the long-sighted game and limited current information. To handle the above challenge, data-driven methods (DDM), such as reinforcement learning (RL), is widely adopted in optimizing online decision-making [3].

In terms of DDM's ability of problem-solving, the probably approximately correct theorem demonstrates that the solution provided by DDM approximates the optimal solution with probability $1-\delta$ [4]. The approximation error is bounded by ε, and the algorithm complexity is positively correlated to $O(n, 1/\varepsilon, 1/\delta)$. Two typical methods are used in the approximation: deep neural network (DNN) reduces generalization risk in global learning space (GLS) via extensive sample training, and it has been widely applied in the fields of image processing [5]. It is also used to estimate the future costs onilne in the energy management of networked MGs [6]. RL, as another method that has drawn rapidly increasing attention in power systems, is applied to study action strategy based on the environment feedback to maximize the expected benefits, such as battery management [7], joint bidding [8], pricing plan Selection [9], and data attack evaluation [10]. Ref. [11], [12] applied RL in demand response, and several works used the RL algorithm in the energy management of MG [13]–[16].

However, existing RL algorithms have the drawbacks of low efficiency, model uncertainty, and difficulty in solving complex cooperative tasks, especially the DOED problem with coupling

Manuscript received 4 April 2020; revised 18 November 2020 and 7 April 2021; accepted 31 May 2021. Date of publication 24 June 2021; date of current version 22 June 2023. This work was supported in part by the National Natural Science Foundation of China under Grant U22B20102 and in part by Harvard Global Institute. Paper no. TPWRS-00535-2020. *(Corresponding author: Tianguang Lu.)*

Tianguang Lu is with the School of Electrical Engineering, Shandong University, Jinan 250061, China (e-mail: tlu@sdu.edu.cn).

Ran Hao is with the State Grid Coporation of China, Beijing 100000, China (e-mail: haoransjtu@sjtu.edu.cn).

Qian Ai is with the Shanghai Jiao Tong University, Shanghai 200240, China (e-mail: aiqian@sjtu.edu.cn).

Hongying He is with the Hunan University, Changsha 410082, China (e-mail: lhx20070322@sina.com).

Color versions of one or more figures in this article are available at https://doi.org/10.1109/TPWRS.2021.3092220.

Digital Object Identifier 10.1109/TPWRS.2021.3092220

objectives and global operation constraints. Limited by various operation constraints, the learning space in the power system is not a general multi-dimensional space, but a partial learning space (PLS). RL-based dispatch should ensure the security and reliability of the system, that is, to satisfy the baseline requirements of PLS, followed by multiple objectives such as economic operation, improving user comfort and increasing RES penetration. Furthermore, learning samples can only be obtained by simulation or collecting historical operation data, and it is difficult to obtain massive learning samples throughout the whole PLS. Appropriate RL methods should ensure baseline requirements and reduce the generalization risk simultaneously, based on limited samples. However, statistical learning theory can only guarantee that DDM approaches the optimal or near-optimal solution of GLS under the premise of massive samples, but fails it to 100% satisfy all baseline operation constraints of PLS. In addition, multiple objectives, local and global constraints of networked MGs are difficult to be integrated into the RL-based algorithms. The author in [17] proposed a fully decentralized RL scheme for the online dispatch of MGs, where no direct interaction among MGs exists. A multi-agent RL algorithm for fully cooperative tasks was proposed in [18]. However, each agent has to acquire their complete objective and no global constraint is considered satisfied collaboratively.

This paper applies a hierarchical framework to improve collaborative learning efficiency for networked MGs and solve the above problems. This paper combines RL and function approximation to achieve a Markov policy decision because function approximation is proven to be an effective way to construct an unbiased estimation of the basis function [19]. In addition, by embedding knowledge from specific domains into RL, knowledge-guiding and data-driven model (KDM) offer a promising way to ensure baseline requirements and improve learning efficiency with limited data.

Incorporated with domain knowledge, hierarchical reinforcement learning (HRL) is developed to solve the problem of DOED for networked MGs. The highlights of this paper are fourfold:

1) The proposed HRL offers a subtle blend of immediate and future rewards, which provides a long-term perspective for DOED. It has highly-improved performance in supporting the long-term operation and optimizing the long-term operation cost.

2) The learning dimension is reduced by the hierarchical design, consequently, the required sample size and computational complexity of the learning are also greatly reduced.

3) Domain knowledge narrows partial learning space to a feasible one and avoids baseline violations, which ensures the feasibility of the operation strategies.

4) The proposed HRL can be implemented online in a model-free distributed manner. Local models do not need to be unified or coupled. Based on learning transfer, the HRL not only learns the optimal strategy offline but also modifies the existing strategy online without prior coordination. Thus, the proposed HRL has extensive model applicability.

The rest of the paper is organized as follows. Taking the global tie-line power constraints into consideration, Section II transforms the basic real-time problem into an integrated KDM model, which facilitates online RL and the coordination of all MGs. Section III develops an online efficient HRL with two policies to optimize networked MGs in a long-term perspective. Section IV provides a convergence proof of the online learning iteration. Finally, Section V shows online multi-period simulation results in an actual multi-MG system to demonstrate the feasibility of HRL and its superiority in computational complexity and long-term economy compared with similar algorithms.

II. Problem Formula of DOED

A. Objective Function of Networked Microgrids

The economic objective of managing networked MGs is to reduce the overall operation costs by coordinating all MGs. The objective consists of three parts: the settlement of trading power C_j^T, dispatchable generation cost $C_{j,i}^U$, and integrated charging/discharging cost C_j^B of BESS.

$$\min_{\substack{P_j^T, P_i^U P_j^B \\ j\in\mathcal{J}, i\in\mathcal{I}_j, t\in\mathcal{T}}} \sum_{t\in\mathcal{T}} \left\{ \sum_{j\in\mathcal{J}} \left[C_j^T \left(P_j^T(t)\right) + \sum_{i\in\mathcal{I}_j} C_{j,i}^U \left(P_{j,i}^U(t)\right) + C_j^B \left(P_j^B(t), SOC_j(t)\right) \right] \right\} \tag{1}$$

where $\mathcal{J}$, $\mathcal{I}_j$ and $\mathcal{T}$ are set of MGs, dispatchable units in MG *j* and time slots, respectively. $P_j^T(t)$ is the total trading power of MG *j*. $P_{j,i}^U(t)$ denotes the power output of dispatchable unit *i* in MG *j*, such as gas turbines or diesel generators. We settle the trading power by real-time price, and define the costs of dispatchable units as a quadratic function of power output:

$$C_j^T \left(P_j^T(t)\right) = \rho(t) \cdot P_j^T(t) \tag{2}$$

$$C_{j,i}^U \left(P_{j,i}^U(t)\right) = a_{j,i}^U \cdot \left(P_{j,i}^U(t)\right)^2 + b_{j,i}^U \cdot P_{j,i}^U(t) + c_{j,i}^U \tag{3}$$

in which $\rho(t)$ denotes the time-of-use price in time slot *t*. $a_{j,i}^U$, $b_{j,i}^U$, and $c_{j,i}^U$ are the cost coefficients of the dispatchable unit *i* in MG *j*.

The integrated BESS costs combine the equivalent value of charge / discharge power and degradation costs, given by

$$C_j^B \left(\boldsymbol{P}_j^B(t), SOC_j(t)\right) = \left(u_j(t-1) - \sigma_j^{dis}\right) \frac{P_j^{B,dis}(t)}{\eta_j^{dis}} - \left(u_j(t-1) + \sigma_j^{ch}\right) \frac{P_j^{B,ch}(t)}{\eta_j^{ch}} \tag{4}$$

where $\boldsymbol{P}_j^B = \{P_j^{B,dis}, P_j^{B,ch}\}$ is discharging and charging power pair of BESS. The following sections use $P_j^B = P_j^{B,dis} - P_j^{B,ch}$ to represent the output of BESS. σ_j^{dis} and σ_j^{ch} are degradation rates of discharging and charging, respectively. According to Ref. [20], integrated BESS costs can be derived by two functions with respect to the state of charge SOC_j. η_j^{ch} and η_j^{dis} denote the charge or discharge efficiencies, respectively. $u_j(t)$ denotes the average unit cost estimation of residual energy in BESS. It provides a basis for online dispatch of BESS.

$SOC_j(t)$ and $u_j(t)$ can be updated by the iterations below:

$$SOC_j(t+1) = SOC_j(t) - \frac{P_j^{B,dis}(t)}{Q_j \cdot \eta_j^{dis}} + \frac{P_j^{B,ch}(t)}{Q_j \cdot \eta_j^{ch}} \tag{5a}$$

$$u_j(t+1) \approx u_j(t) + \frac{\left(\rho(t)+\sigma_j^{ch}\right) P_j^{B,ch}(t)/\eta_j^{ch} - \left(u_j(t)-\sigma_j^{dis}\right) P_j^{B,dis}(t)/\eta_j^{dis}}{SOC_j(t) \cdot Q_j} \tag{5b}$$

where Q_j is the capacity of the BESS in MG *j*. In (5b), $(\rho(t) + \sigma_j^{ch})P_j^{B,ch}(t)/\eta_j^{ch}$ denotes the real-time charging cost, and $(u_j(t) - \sigma_j^{dis})P_j^{B,dis}(t)/\eta_j^{dis}$ denotes the real-time discharging cost of BESS.

B. Constraints

This paper considers that the constraints in system operation include global constraint - total tie-line power constraints, and local constraints for each MG. The local constraints include balance constraints, capacity constraints, and BESS operation constraints. These constraints are provided in (6)–(11) below.

In this paper, we assume that MGs can purchase power or feed surplus power back to a bus. $P^T(t)$ is defined as the total real-time tie-line power of the networked MGs. The bidirectional trading power between the bus and utility grid should not deviate too much from the day-ahead planning to restrict excessive uncertainty of MGs transferred to the grid. Thus, we set the lower and upper bounds of the total trading power as $\underline{P}^T(t)$ and $\bar{P}^T(t)$ respectively. Global tie-line power constraints can be written as

$$\sum_{j\in\mathcal{J}} P_j^T(t) \in \left[\underline{P}^T(t), \bar{P}^T(t)\right] \tag{6}$$

For each MG, the local generation, including dispatchable unit outputs $\sum_{i\in\mathcal{I}_j} P_{j,i}^U(t)$, renewable power output $P_j^{RES}(t)$, and BESS discharging/charging power $P_j^B(t)$, plus the power interacting with the external system should always equal the total load $P_j^L(t)$ in this region.

$$\sum_{i\in\mathcal{T}_j} P_{j,i}^U(t) + P_j^{RES}(t) + P_j^T(t) + P_j^B(t) = P_j^L(t) j \in \mathcal{J} \tag{7}$$

The capacity constraints restrict the generation bounds of dispatchable units.

$$\underline{P}_{j,i}^U \le P_{j,i}^U(t) \le \bar{P}_{j,i}^U \tag{8}$$

where $\underline{P}_{j,i}^U$ and $\bar{P}_{j,i}^U$ denote the lower and upper bounds of unit *i*, respectively. They can be derived by unit characteristics and reserve requirements provided by day-ahead scheduling.

The operation constraints of BESS are incorporated by charging or discharging state and power restrictions.

$$P_j^{B,dis}(t) \cdot P_j^{B,ch}(t) = 0 \tag{9}$$

$$0 \le P_j^{B,dis}(t) \le \min\left\{\bar{P}_j^{B,dis}, Q_j\left(SOC_j(t) - \underline{SOC}_j\right)\right\} \tag{10}$$

$$0 \le P_j^{B,ch}(t) \le \max\left\{\underline{P}_j^{B,ch}, Q_j\left(SOC_j(t) - \overline{SOC}_j\right)\right\} \tag{11}$$

C. Integrated KDM Learning Model

General operation constraints are difficult to be applied in establishing PLS by multiple local learners. Therefore, we convert these local constraints (6)–(11) into rules by embedding knowledge from specific domains in order to enhance the generalization accuracy of our RL model. In summary, combining the global objective of networked MG (1) and PLS guided by domain knowledge set, the DOED problem can be recast as the following KDM problem:

$$(\mathbf{KDM}) \min_{\boldsymbol{x}_j \triangleq \{P_j^T, P_{i,j}^U, \boldsymbol{P}_j^B | i\in\mathcal{I}_j\}} C_j = C_j^T\left(P_j^T(t)\right) + \sum\nolimits_{i\in\mathcal{I}_j} C_{j,i}^U\left(P_{j,i}^U(t)\right) + C_j^B\left(\boldsymbol{P}_j^B(t), SOC_j(t)\right)$$

$$s.t. \left\|\boldsymbol{f}_j(\boldsymbol{x}_j, \boldsymbol{\theta}_j), \boldsymbol{\kappa}_j(\boldsymbol{x}_j, \boldsymbol{\zeta}_{\kappa_j})\right\|_\sigma \le \boldsymbol{k}_j \tag{12}$$

where C_j is defined as the objective of MG *j*. Vector $\boldsymbol{\theta}_j$ is the learning parameter set of **KDM**. $\boldsymbol{f}_j$ is a hypothesis space of learning parameters and decision variables. $\boldsymbol{\kappa}_j$ denotes the knowledge rule set of MG *j*. $\|\boldsymbol{f}_j, \boldsymbol{\kappa}_j\|_\sigma$ is the knowledge-constrained operator. The knowledge constraints in $\|\boldsymbol{f}_j, \boldsymbol{\kappa}_j\|_\sigma$ are modeled as chance constraints which may not be satisfied under certain circumstances. $\boldsymbol{k}_j$ denotes the threshold vector for all knowledge rules in MG *j*. $\boldsymbol{\zeta}_{\kappa_j} \in [0, 1]$ is a set of hyperparameter for knowledge rules set $\boldsymbol{\kappa}_j$, which indicates the data confidence in each rule. If an element in $\boldsymbol{\zeta}_{\kappa_j}$ approaches 0, then it means that the confidence level of the corresponding rule becomes lower. In this paper, all elements in $\boldsymbol{\zeta}_{\kappa_j}$ are set to 1 because all constraints are certain to exist in actual operation.

$$\boldsymbol{\kappa}_j = \begin{cases} \kappa_{j,1}: \sum\limits_{i\in\mathcal{I}_j} P_{j,i}^U(t) + P_j^B(t) + P_j^{RES}(t) + P_j^T(t) = P_j^L(t) \\ \kappa_{j,2}: \text{if } P_{j,i}^U(t) > \bar{P}_{j,i}^U, \text{ then } P_{j,i}^U(t) = \bar{P}_{j,i}^U \\ \kappa_{j,3}: \text{if } P_{j,i}^U(t) < \underline{P}_{j,i}^U, \text{ than } P_{j,i}^U(t) = \underline{P}_{j,i}^U \\ \kappa_{j,4}: \text{if } P_{j,i}^T(t) > \bar{P}_{j,i}^T, \text{ then } P_{j,i}^T(t) = \bar{P}_{j,i}^T \\ \kappa_{j,5}: \text{if } P_{j,i}^T(t) < \underline{P}_{j,i}^T, \text{ then } P_{j,i}^T(t) = \underline{P}_{j,i}^T \\ \kappa_{j,6}: \text{if } P_j^{B,dis}(t) \ge 0 \text{ or} \\ \quad P_j^{B,dis}(t) \le \min\{\bar{P}_j^{B,dis}, Q_j(SOC_j(t) - \underline{SOC}_j)\} \\ \quad \text{then } P_j^{B,dis}(t) = 0 \text{ or} \\ \quad P_j^{B,dis}(t) = \min\{\bar{P}_j^{B,dis}, Q_j(SOC_j(t) - \underline{SOC}_j)\} \\ \kappa_{j,7}: \text{if } P_j^{B,ch}(t) \le 0 \text{ or} \\ \quad P_j^{B,ch}(t) \ge \max\{P_j^{B,ch}, Q_j(SOC_j(t) - \overline{SOC}_j)\} \\ \quad \text{then } P_j^{B,ch}(t) = 0 \text{ or} \\ \quad P_j^{B,ch}(t) = \max\{P_j^{B,ch}, Q_j(SOC_j(t) - \overline{SOC}_j)\} \end{cases} \tag{13}$$

The KDM problem is designed for each MG, implying that only local constraints are handled by the rule-set. $\|\boldsymbol{f}_j(\boldsymbol{x}_j, \boldsymbol{\theta}_j), \boldsymbol{\kappa}_j(\boldsymbol{x}_j, \boldsymbol{\zeta}_{\kappa_j})\|_\sigma \le \boldsymbol{k}_j$ describes a mapping set acting on space $\boldsymbol{f}_j(\boldsymbol{x}_j, \boldsymbol{\theta}_j)$, which is regarded as the constraints or knowledge rule-set in this paper. The knowledge rule-set $\boldsymbol{\kappa}_j$ of each MG is given in (13)

In this ruleset, $\kappa_{j,1}$ is built according to (7). $\kappa_{j,2}$ and $\kappa_{j,3}$ are generated by (8). $\kappa_{j,4}$ and $\kappa_{j,5}$ are derived from (6). $\kappa_{j,5}$

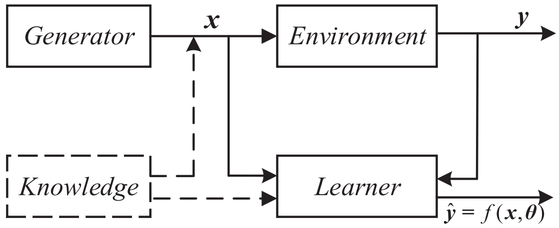

Fig. 1. Knowledge-guiding and data-driven learning problem.

and $\kappa_{j,6}$ are given by (10) and (11) The proposed hierarchical structure establishes a one-to-one correspondence between variables. Once a variable is determined, another condition can be determined according to the constraint conditions. Therefore, when there exist tight constraints, we don't need to discrete the learning space and set the basis functions.

According to the topological geometry [21], the joint probability distribution of a practical problem can be regarded as an unknown manifold in a high-dimensional space, while the knowledge set κ_j is a low-dimensional manifold, and is homeomorphic to the unknown high-dimensional manifold. At this time, learning can be regarded as a manifold reconstruction. On the low-dimensional manifold corresponding to the knowledge, the reconstructed manifold constantly approaches the real high-dimensional manifold by continuously embedding high-dimensional samples.

According to Fig. 1, generators are utilized to generate random samples or collect real-time data. The environment maps input vector $\boldsymbol{x}$ to output vector $\boldsymbol{y}$. Learner provides a universal function $f(\boldsymbol{x}, \boldsymbol{\theta})$ to approach $\boldsymbol{y}$. The knowledge model formalizes experience and embeds it into learning process.

The optimal solution is provided by the data-driven models may still contravene the basic knowledge of the domain and the baselines of learning space, which may result in higher generalization error. KDM establishes PLS and avoids the violation of operation constraints by embedding domain knowledge, which helps to reduce the generalization error.

III. Efficient Hierarchical Reinforcement Q-Learning

HRL is proven to be a promising approach in extending traditional RL to handle complex problems by decomposing learning tasks. HRL requires careful task-specific design and off-policy training to reduce the demand for sample size, making it more feasible in real scenarios.

A. Combine-Then-Adapt Mechanism

Here, the combine-then-adapt (CTA) mechanism is involved in the HRL to coordinate power interaction among MGs. The global tie-line constraint $\sum_{j\in\mathcal{J}} P_j^T(t) \in [\underline{P}^T(t), \bar{P}^T(t)]$ can be learned synchronously by introducing the penalty item in the reward function.

$$r_j(s_j, a_j) = -[C_j(s_j, a_j) + \upsilon_j \Upsilon_j(s_j, a_j) \tag{14}$$

where $C_j(s_j, a_j)$ is the integrated operation cost of each MG given by **KDM** (12). $\Upsilon_j(s_j, a_j)$ is a Lagrange penalty item for violating global constraints (6), and υ_j is the penalty coefficient. A variant of the logarithmic barrier function is employed here to handle the global inequality constraints according to [22].

$$\Upsilon_j(s_j, a_j) = \begin{cases} \frac{1}{Y} \ln \frac{1}{1-\left(\frac{2\sum P_j^T - \bar{P}^T - \underline{P}^T}{\bar{P}^T - \underline{P}^T}\right)^2}, \sum P_j^T \in [\underline{P}^T, \bar{P}^T] \\ M_{Big}, o.w. \end{cases} \tag{15}$$

where $\sum P_j^T$ is the total trading power of the networked MG system. It can be obtained by each MG via consensus-based iteration [23]. *Y* is an approaching parameter, and M_{Big} is set to a big positive integer.

The CTA method first combines the global penalty item in the local objectives. Then, each MG decides its policy and feeds the results back to the combined penalty item. The CTA mechanism ensures the power balance of the whole system.

B. Hierarchical RL of Two Policies

In the dispatch learning of the networked MGs, a contradiction exists between limited learning samples and excessive related factors. The optimal variables of an MG depend on the states of other MGs. But, the distribution of operation data is extremely uneven in the entire learning space. Historical data that can be trained are insufficient if all related factors are considered in one policy decision.

Therefore, according to the correlation analysis in *Appendix C (part A)*, we extend the general RL to HRL by separating learning space into a bilayer learning spaces [24], with a low-level action policy, and a high-level action policy. The high-level policy operates at a coarser level of abstraction and reduces the dimension of low-level learning space [25]. The overall framework design of HRL is demonstrated in the figure below.

Coordinated by the CTA mechanism, HRL is carried out independently by each MG. The sequence flow of HRL is illustrated as follows:

1. At each time slot *t*, the environment provides a local observation set $s_j = \{\rho, P_j^L - P_j^{RES}, SOC_j\}$. Initialize action set $a_j = \{P_j^T, P_j^B, \sum P_{j,i}^U | i \in \mathcal{I}_j\}$ and its immediate reward r_j, consisting of the global constraints' penalty $-\upsilon_j \Upsilon_j$ and intrinsic reward $-C_j$.

2. $a_j^{high} = \{P_j^B + \sum P_{j,i}^U\}$ is set as the high-level decision variable. The basic function weights $\boldsymbol{\theta}_j^{high}$ of high-level *Q* network with reward r_j are trained as the high-level decision variable to determine the total generation output of MG *j*.

3. Each MG calculates *Q* values with different actions of various states and selects $a_j^{high}(t)$ with the maximum *Q* as the high-level best-action a_j^{high*} of s_j.

4. Once the observation set s_j is given, transform the knowledge rules for P_j^T, (*i.e.* $\kappa_{j,4}$ and $\kappa_{j,5}$), to the rules for the high-level variable according to the power balance rule $\kappa_{j,1}$. If $a_j^{high*} \in \boldsymbol{f}_j$, knowledge rules $\kappa_{j,2} \sim \kappa_{j,7}$ restrict a_j^{high*}. Then, the best-action policy of the high-level learning a_j^{high*} can be obtained.

5. Based on the qualitative and quantitative correlation analysis, each MG defines $a_j^{low} = \{P_j^B, \sum P_{j,i}^U | i \in \mathcal{I}_j\}$ as the low-level decision variable set. Then, each MG trains low-level decision variables to determine the BESS generation P_j^B. In this process, $-C_j$ is served as additional state observation.

6. Each MG calculates *Q* values with different actions of state s_j and selects $P_j^B(t)$ with the maximum *Q* as the best-action P_j^{B*}.

7. According to the rule $\kappa_{j,1}$, calculate $\sum_{i \in \mathcal{I}_j} P_{j,i}^{U*} = a_j^{high*} - P_j^{B*}$. If $a_j^{low*} = \{P_j^{B*}, \sum_{i \in \mathcal{I}_j} P_{j,i}^{U*}\} \in \boldsymbol{f}_j$, the knowledge rules $\kappa_{j,2}, \kappa_{j,3}, \kappa_{j,6}$ and $\kappa_{j,7}$ restrict the irregular action in the low-level and re-calculate the other action according to $\kappa_{j,1}$. Then, the best-action policy of the low-level learning a_j^{low*} can be obtained.

8. The action is executed and the next slot state $s_j(t+1)$ is updated.

The high level selects the local states $s_j^{high} = \{P_j^L - P_j^{RES}, SOC_j\}$ as learning inputs of MG *j*, which is highly correlated with its local decision variables $P_j^B + \sum P_{j,i}^U$ to reduce the algorithm computational complexity. After obtaining the total dispatchable generation, the low level selects the local states $s_j^{low} = \{P_j^B + \sum P_{j,i}^U, SOC_j\}$ as the learning inputs. Knowledge rules $\kappa_{j,2} \sim \kappa_{j,7}$ jointly constrain the high-level decision variables and ensure that the low-level learning has feasible solutions.

The proposed HRL stands out from previous methods by being generally applicable and data-efficient. It reduces the learning dimension by hierarchical learning. It also avoids local data communication. The high-level policy instructs the low-level policy to reach specific states, which refines the original learning space and allows the low-level policy to easily learn from prior off-policy experiences easily. In addition, KDM decreases RL uncertainty and ensures learning baselines.

C. *Off-Policy Temporal Difference Q-Learning*

In practical interactive energy management of networked MGs, RL has superiority in the following points: *a*. improving priori knowledge accuracy in an uncertain environment. *b*. a fully distributed online solution method for MGs. *c*. increasing sensitivity to the parameters in the reward function. *d*. long-sighted decision without any future state observation. Thus, we define an instant state-action pair (s_j, a_j) and *Q* function below:

$$Q_{t+1,j}(s_j, a_j) = Q_{t,j}(s_j, a_j) + \alpha_t \left[r_{t,j}(s_j, a_j) + \gamma \max_{a'_j} Q_{t,j}(s_j{}', a_j{}') - Q_{t,j}(s_j, a_j) \right] \tag{16}$$

where $(s_j{}', a_j{}')$ denotes the state-action pair in the next cycle. $r_{t,j}(s_j, a_j)$ denotes immediate reward if MG *j* with state s_j prefers to take action a_j, γ is an exploration rate, and α_t is the discount factor. This *Q* function estimates the expectation of long-term cost. This *Q* function estimates the expectation of long-term cost, which can be proven in *Appendix A*.

Artificial neural networks, such as backpropagation networks and DNN, approximate complex system models by neural networks [26]. In the neural network, there is no temporal relationship between samples, and real-time samples are easily affected by historical samples, making it suitable for centralized day-ahead dispatch rather than distributed online dispatch. Besides, day-ahead optimization involves more scheduling cycles, and the demand for sampling number increases exponentially. Thus, temporal difference *Q*-learning is used to estimate the maximum future rewards by enumerating features in a series of discrete states to improve the generalization ability of modeled *Q* in a distributed way. It develops an expert system, where optimal actions can be selected by generalizing samples.

Based on the evolution equation (16), the *Q* function can be adjusted online. The *Q* function is described by a linear combination of Gaussian random functions, with the ability of perfect generalization, serving as radial basis function $\varphi_{l,j}$ (RBF).

$$Q_{t,j}(s_j, a_j) = \sum\nolimits_{l=1}^{m} \varphi_{t,j}(s_j, a_j)\theta_{l,j} = \varphi_j{}^T \boldsymbol{\theta}_j \tag{17}$$

where $\boldsymbol{\theta}_j \in \mathbb{R}^m$ is an approximation parameter vector, which is required to be optimized, and *m* is the number of state features. For ease of notation, we regard $\boldsymbol{\varphi}_{t,j}(s_j, a_j)$ as $\boldsymbol{\varphi}_j(t)$ in the subsequent parts.

We define **x** as the state point and assume $\boldsymbol{x}^{Cen}$ as the RBF center. Gaussian function is used as RBF, which is described as

$$\varphi_{t,i} = G_i(x, x^{Cen}) = exp\left(-\left\|x - x^{Cen}\right\|^2 / 2\sigma_i{}^2\right) \tag{18}$$

The centers of Gaussian functions are set to be evenly distributed in the learning space. In this paper, each variable is equally divided into 20 discrete values. Besides, the variances of Gaussian functions can be determined by the empirical formula:

$$\sigma_i = \frac{c_{\max}}{\sqrt{2h}}, i = 1, 2, \ldots, h \tag{19}$$

where, c_{max} is the maximum distance of two centers, and *h* denotes the number of centers. Based on the RBF, *Q* functions can approximate an arbitrary hyperplane. A schematic diagram of applying a Gaussian function to learn the *Q* function in one-dimensional state space is shown in Fig. 3.

Projected *Bellman* error B_j can be used to evaluate the approaching performance [27]. Temporal difference (TD) error ε_j quantifies the gap between the real and estimated rewards.

$$B_j = E[\varepsilon_j(t)\boldsymbol{\varphi}_j(t)]^T E[\boldsymbol{\varphi}_j(t)\boldsymbol{\varphi}_j(t)^T]^{-1} E[\varepsilon_j(t)\boldsymbol{\varphi}_j(t)] \tag{20}$$

$$\varepsilon_j(t) = r_j(t) + \gamma \hat{\boldsymbol{\varphi}}_j(t)\theta_j(t) - \boldsymbol{\varphi}_j(t)^T \boldsymbol{\theta}_j(t) \tag{21}$$

where $\hat{\boldsymbol{\varphi}}_j(t)\boldsymbol{\theta}_j(t)$ is the optimal approximation of the future *Q* function $\max Q_{t,j}(s_j{}', a_j{}')$. Therefore, $\hat{\boldsymbol{\varphi}}_j(t)$ can be estimated by

$$\hat{\boldsymbol{\varphi}}_j(t) \approx arg \max_{s'_j, a'_j} \boldsymbol{\varphi}_j(s'_j, a'_j)\boldsymbol{\theta}_j \tag{22}$$

The gradient of *Bellman* error can be obtained based on [19].

$$\nabla B_j / 2 = -\mathbb{E}[\varepsilon_j(t)\boldsymbol{\varphi}_j(t)^T] + \gamma \mathbb{E}[\hat{\boldsymbol{\varphi}}_j(t)\boldsymbol{\varphi}_j(t)^T]\boldsymbol{\omega}_j^*(t) \tag{23}$$

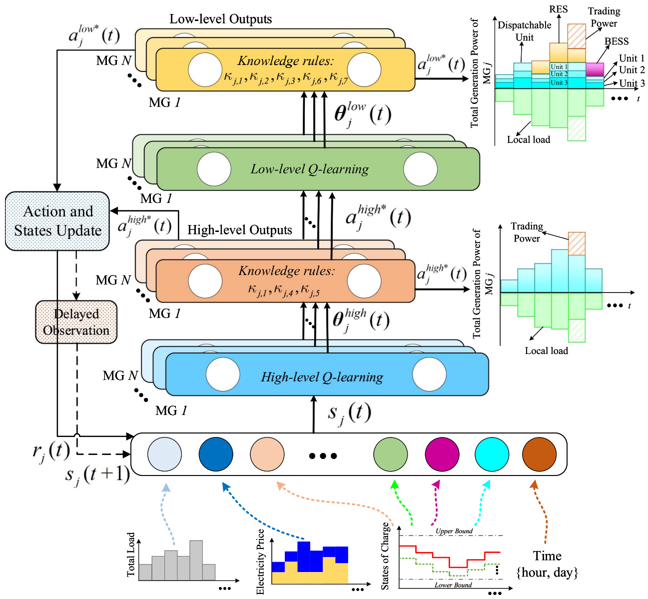

Fig. 2. The overall framework design of HRL.

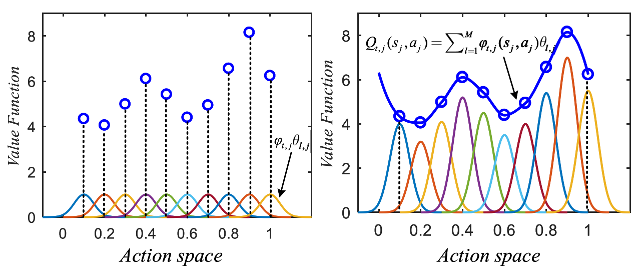

Fig. 3. Schematic diagram of RBF approximating one-dimensional Q function.

where $\boldsymbol{\omega}_j^*(t)$ is an auxiliary variable, that is equal to $\mathbb{E}[\boldsymbol{\varphi}_j(t)\boldsymbol{\varphi}_j(t)^T]^{-1}\mathbb{E}[\varepsilon_j(t)\boldsymbol{\varphi}_j(t)]$.

The *Greedy-Go* strategy shows an impressive performance in the RL update [28]. Here, we use $\boldsymbol{\omega}_j(t)$ to estimate $\boldsymbol{\omega}_j^*(t)$, and regard the gradient of projected *Bellman* error as the update reference:

$$\boldsymbol{\theta}_j(t+1) = \theta_j(t) + \alpha_t\left(\varepsilon_j(t)\boldsymbol{\varphi}_j(t) - \gamma\boldsymbol{\omega}_j(t)^T\boldsymbol{\varphi}_j(t)\hat{\boldsymbol{\varphi}}_j(t)\right) \tag{24}$$

$$\boldsymbol{\omega}_j(t+1) = \boldsymbol{\omega}_j(t) + \beta_t\boldsymbol{\varphi}_j(t)\left(\varepsilon_j(t) - \boldsymbol{\varphi}_j(t)^T\boldsymbol{\omega}_j(t)\right) \tag{25}$$

where β_t is the learning rate of the auxiliary variable $\boldsymbol{\omega}_j^*(t)$.

Before online adaptive learning, pre-learning is essential in improving dispatch accuracy. Here, an offline learning, guided by domain knowledge, is established and transferred to initialize the online HRL. The pre-learning procedures can be summarized in **Algorithm 1**. Step 4 and step 7 in the offline learning embed knowledge rules in high-level and low-level, respectively.

Initialized by the weights in pre-learning, online HRL adaptively adjusts dispatch schemes in response to any changes in cost models, topology, or MG networking, where each MG only needs to communicate trading power with neighbors.

Algorithm 1: Distributed Pre-Learning of Hierarchical Reinforcement Learning.

Input: Reward-action curves in each discrete state in two levels;
Output:Pre-trained RBF weights $\boldsymbol{\theta}_j(t)$ of two levels. Best-action policies;
1: Discretize low-level and high-level state spaces.
2: Distribute the two-level RBFs in action space at each state.
3: Calculate high-level rewards, and set RBF weights $\boldsymbol{\theta}_j(t)$.
4: Search the maximum Q and determine the optimal high-level action with the guidance of knowledge.
5: Describe the high-level action policy (optimal action-state) via RBF.
6: Repeat steps 3-5 for low-level pre-learning based on low-level Q values.
7: Return **Output**.

Algorithm 2: Distributed Online HRL of Two Policy with Gaussian Function Approximation.

Input: Pre-trained RBF weights $\boldsymbol{\theta}_j(t)$, learning rate α_t and β_t, and doubly stochastic matrix $\boldsymbol{W}$ for consensus iteration.;
Output: Updated RBF weights $\boldsymbol{\theta}_j(t)$ and real-time action $a_j(t)$ of each MG j;
Set initial states $\boldsymbol{\omega}_j(0)$, $s_j(0)$, initial RBFs $\boldsymbol{\varphi}_j$ and pre-learned RBF weights $\boldsymbol{\theta}_j$.
for *each cycle t = 1 to T:* **do**
  for *each MG j = 1 to N* **do**
    Take high-level action $a_j^{high}(t)$ according to high-level action policy.
    Take low-level action $a_j^{low}(t)$ according to its action policy and $a_j^{high}(t)$.
    Observe the additional state observation $-C_j$ and immediate reward r_j.
    Calculate the TD error of each level ε_j by (21).
    Calculate the approximate $\hat{\boldsymbol{\varphi}}_j(t)$ based on equation (22).
    Update weights $\boldsymbol{\theta}_j^{low}$, $\boldsymbol{\theta}_j^{high}$ and auxiliary variables $\boldsymbol{\omega}_j^{high}$, $\boldsymbol{\omega}_j^{low}$ by (24), (25).
    Calculate the maximum Q values and corresponding best actions.
    If $a_j^{low*}, a_j^{high*} \in \boldsymbol{f}_j$, the knowledge rule-set $\boldsymbol{\kappa}_j$ acts on the two-level action policies.
    Update the two-level action policies.
    Given $\sum P_{j,i}^U$, MG optimizes the outputs of local dispatchable units $P_{j,i}^U$.
  end
  Each MG executes its dispatch decision.
end
Return **Output**.

The main online procedures are listed in **Algorithm 2**.

According to **Algorithm 2**, the online iteration for a time slot includes action decision, knowledge guiding, new reward perception, and RBF weights adjustment. Define M as the discretization number of each input variable. Since the low-level RL contains two-state inputs, RBF φ_j and corresponding approximation weight $\boldsymbol{\theta}_j$ are distributed on the two-dimensional plane. Therefore, the computational complexity of low-level learning is $O(M^2)$. Considering the two-level, the computational complexity of one-cycle dispatch is $O(2M^2)$. Consider N MGs in T-slot dispatch, the computational complexity is $O(2M^2 \cdot TN)$. Without domain knowledge, $s_j =$

$\{\rho, P_j^L - P_j^{RES}, SOC_j\}$ is the state set of traditional fuzzy *Q*-learning and the learning dimension is equal to 3. Similarly, there are at least two local policies and one global policy to learn $a_j = \{\sum P_j^I | j \in \mathcal{J}, P_j^B, \sum P_{j,i}^U | i \in \mathcal{I}_j\}$. Thus, the computational complexity of tradtional fuzzy *Q*-learning is $O((2N+1)M^3 \cdot T)$. Therefore, the learning efficiency of HRL is much higher than that of the fuzzy *Q*-learning.

IV. Convergence Proof of Online Learning

The convergence of approximation parameters is vital in the learning iteration of networked MGs. *Appendix B* provides general convergence conditions when learning rates are time-varying. This section analyzes the sensitivity of learning rates and provides practical convergence condition to select fixed learning rates, which is more practical.

The definition equation of ε_j is integrated into (24) and (25).

$$\begin{aligned}\boldsymbol{\theta}_j(t+1) &= \boldsymbol{\theta}_j(t) + \alpha_t \boldsymbol{\varphi}_j(t)\left(\gamma \hat{\boldsymbol{\varphi}}_j(t) - \boldsymbol{\varphi}_j(t)\right)\boldsymbol{\theta}_j(t) \\ &\quad - \alpha_t \gamma \hat{\boldsymbol{\varphi}}_j(t) \boldsymbol{\varphi}_j(t)^T \boldsymbol{\omega}_j(t) + \alpha_t r_j(t) \boldsymbol{\varphi}_j(t) \end{aligned} \tag{26}$$

$$\begin{aligned}\boldsymbol{\omega}_j(t+1) &= \boldsymbol{\omega}_j(t) + \beta_t \boldsymbol{\varphi}_j(t)\left(\gamma \hat{\boldsymbol{\varphi}}_j(t) - \boldsymbol{\varphi}_j(t)\right)\boldsymbol{\theta}_j(t) \\ &\quad - \beta_t \boldsymbol{\varphi}_j(t) \boldsymbol{\varphi}_j(t)^T \boldsymbol{\omega}_j(t) + \beta_t r_j(t) \boldsymbol{\varphi}_j(t) \end{aligned} \tag{27}$$

According to ref. [29], [30], combining the two transferred variables into a pair $\boldsymbol{\delta}_i(t+1) = [\boldsymbol{\theta}_j(t+1), \boldsymbol{\omega}_j(t+1)]^T$, the above two equations can be rewritten as follows

$$\begin{aligned}\boldsymbol{\delta}_i(t+1) &= \boldsymbol{\delta}_j(t) + \begin{bmatrix} \alpha_t F_j(t) & -\gamma \alpha_t \hat{G}_j(t) \\ \beta_t F_j(t) & -\beta_t G_j(t) \end{bmatrix} \cdot \boldsymbol{\delta}_j(t) \\ &\quad + \begin{bmatrix} \alpha_t r_j(t) \boldsymbol{\varphi}_j(t) \\ \beta_t r_j(t) \varphi_j(t) \end{bmatrix} \\ &= \boldsymbol{\delta}_j(t) + \left(\boldsymbol{H}_j(t) \cdot \boldsymbol{\delta}_j(t) + \boldsymbol{h}_j(t)\right) \end{aligned} \tag{28}$$

Specifically, the introduced three variable matrices are provided as

$$F_j(t) = \boldsymbol{\varphi}_j(t)\left[\gamma \hat{\boldsymbol{\varphi}}_j(t) - \boldsymbol{\varphi}_j(t)\right]^T \tag{29}$$

$$\hat{G}_j(t) = \hat{\boldsymbol{\varphi}}_j(t) \varphi_j(t)^T, G_j(t) = \boldsymbol{\varphi}_j(t) \boldsymbol{\varphi}_j(t)^T \tag{30}$$

That a point $\boldsymbol{\delta}^0$ satisfying $\boldsymbol{H}_j \cdot \boldsymbol{\delta}^0 + \boldsymbol{h}_j = \mathbf{0}$ exists can be proven. The error vector of the variable pair is defined as

$$\boldsymbol{e}_{\delta,j}(t) \triangleq \boldsymbol{\delta}^0 - \boldsymbol{\delta}_j(t) \tag{31}$$

Substitute (28) into the definition of $\boldsymbol{e}_{\delta,j}(t)$, we can recast the error vector in a recursive form, provided by

$$\begin{aligned}\boldsymbol{e}_\delta(t+1) &= \mathbf{1}_N \otimes \boldsymbol{\delta}^0 - \boldsymbol{\delta}(t+1) \\ &= \mathbf{1}_N \otimes \boldsymbol{\delta}^0 - \left(\boldsymbol{I}_{N_\varphi} + \boldsymbol{H}(t)\right)\boldsymbol{\delta}(t) - \boldsymbol{h}(k) \\ &= \left(\boldsymbol{I}_{N_\varphi} + \boldsymbol{H}(t)\right)\boldsymbol{e}_\delta(t) + \mathbf{1}_N \otimes \boldsymbol{\delta}^0 - \mathbf{1}_N \otimes \boldsymbol{\delta}^0 \\ &\quad - \boldsymbol{H}(t)\mathbf{1}_N \otimes \boldsymbol{\delta}^0 - \boldsymbol{h}(k) \end{aligned} \tag{32}$$

where $\otimes$ represents the Kronecker product. $\boldsymbol{N}_\varphi \in \mathbb{R}^{N \cdot M \times N \cdot M}$. Moreover, $\boldsymbol{H}(t) = diag\{\boldsymbol{H}_1(t), \boldsymbol{H}_2(t), \ldots, \boldsymbol{H}_N(t)\}$, and $\boldsymbol{H}(t) = diag\{\boldsymbol{H}_1(t), \boldsymbol{H}_2(t), \ldots, \boldsymbol{H}_N(t)\}$ is a column vector. According to (31), we also have $\boldsymbol{\delta}(t) = \mathbf{1}_N \otimes \boldsymbol{\delta}^0 - \boldsymbol{e}_\delta(t)$.

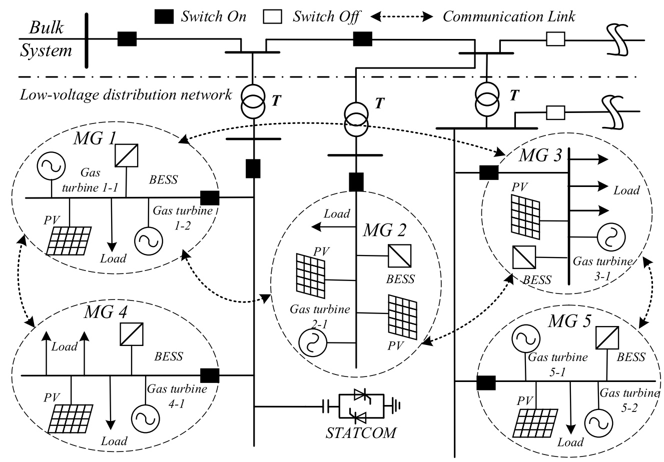

Fig. 4. Test system based on the actual networked MGs system in Qingdao.

In (32), $\boldsymbol{e}_\delta(t+1)$ and $\boldsymbol{\delta}(t)$ are variables because $\boldsymbol{H}(t)\mathbf{1}_N \otimes \boldsymbol{\delta}^0$ and $\boldsymbol{h}(k)$ are constants, and we can obtain that $\boldsymbol{H}(t)\mathbf{1}_N \otimes \boldsymbol{\delta}^0 + \boldsymbol{h}(k) = \mathbf{0}$. The expectation on both sides of (32) can be described as

$$\begin{aligned}&\mathbb{E}\left[\boldsymbol{e}_\delta(t+1)\right] = \left(\boldsymbol{I}_{N_\varphi} + \mathbb{E}\left[\boldsymbol{H}(t)\right]\right)\mathbb{E}\left[\boldsymbol{e}_\delta(t)\right] \\ &= \left(\boldsymbol{I}_{N_\varphi} + \mathbb{E}\left[\boldsymbol{H}_j(t)\right]\right)\mathbb{E}\left[\boldsymbol{e}_{\delta,j}(t)\right] = \left(\boldsymbol{I}_{N_\varphi} + \boldsymbol{H}_j(t)\right)\mathbb{E}\left[\boldsymbol{e}_{\delta,j}(t)\right] \end{aligned} \tag{33}$$

The learning is convergent if the maximum eigenvalue of the transition matrix $(\boldsymbol{I}_{N_\varphi} + \mathbb{E}[\boldsymbol{H}_j(t)])$ is less than 1.

$$\left|1 + \lambda_{\max}\left(\boldsymbol{H}_j\right)\right| < 1, \forall j \in [1, N] \tag{34}$$

The convergence of RL is determined by the maximum eigenvalue of $\boldsymbol{H}_j$. According to the definition, its elements are completely determined by learning parameters α_t, β_t and feature vector $\boldsymbol{\varphi}_j(t)$.Since the Gaussian function is used as the RBF, the modulus of $F_j(t)$, $\hat{G}_j(t)$ and $G_j(t)$ are bounded by the range [0, 1]. Condition (34) is a necessary and sufficient condition for learning parameter design. Therefore, α_t, β_t can be selected small enough to satisfy the necessary condition to guarantee the learning convergence. Although small learning rates can guarantee learning convergence, they will reduce the convergence rate.

V. Case Studies

A. Parameters and Relevant Analysis

The DOED research serves the project of networked MGs in the Sino German Ecological Park, Qingdao, China. The industrial park is mapped to the test case, consisting of five MGs, as depicted in Fig. 4. A sparsely distributed communication diagram is also provided. A large number of rooftop photovoltaic devices have been installed dispersedly in each MG. Industrial gas turbines LM6000 PF and BESS NP200-12 are also planned to be configured in this park. The typical weekly load and renewable generation data in March 2019 are selected as the learning environment. Equivalent parameters of generators

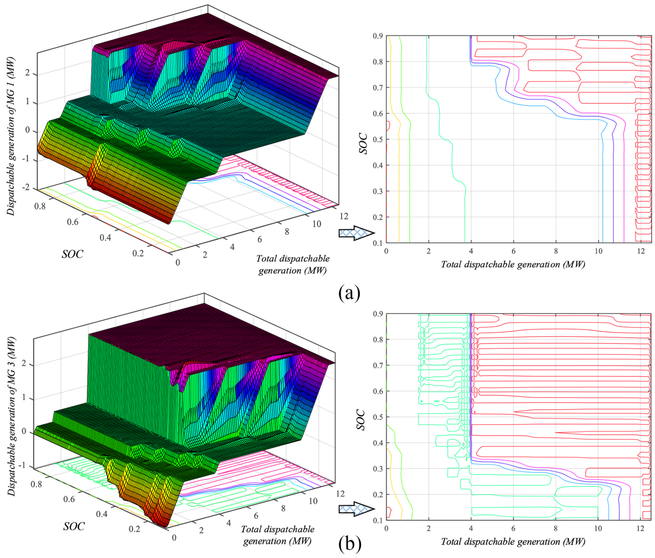

Fig. 5. High-level best-action policies of two representatives MGs. (a) MG 1. (b) MG 3.

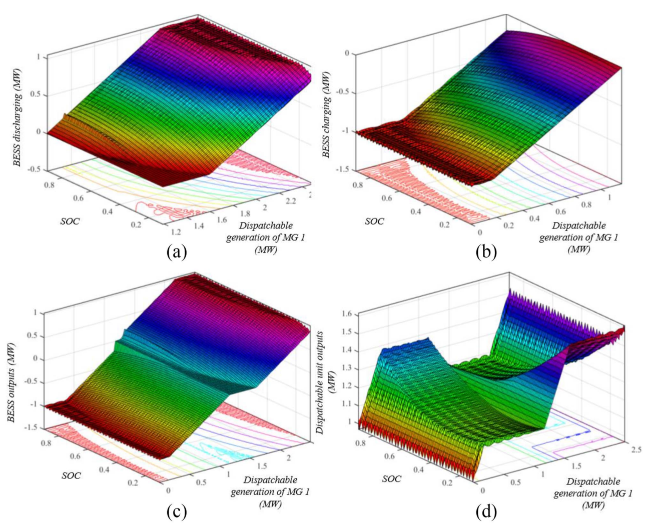

Fig. 6. The best-action policy of MG 1 in low-level RL. (a) BESS discharging power. (b) BESS charging power. (c) Combined BESS outputs. (d) Dispatchable units' outputs.

and BESS are obtained by the field survey and are given in Appendix E.

B. Trained Action Policies and Dispatch Results

This part provides the simulation results of our proposed HRL after pre-learning and one-week online scheduling. The high-level and low-level best-action policies are presented in Fig. 5 and Fig. 6, respectively.

The high-level policy provides the total dispatchable generation of MG j ($P_j^L - P_j^{RES}$), whereas the low-level policy decides BESS and dispatchable units' outputs. According to Fig. 5, the MG with low dispatchable generation cost, e.g. MG 3, has priority over the MG with high dispatch cost, e.g. MG 1, in the power supply. In Fig. 6, when SOC is low, the BESS tends to generate less power due to the high marginal cost. Meanwhile, this will lower the total dispatchable generation of the MG. A potential trade-off exists in the low-level power-sharing between BESS and units because the total dispatchable generation is given by the high layer.

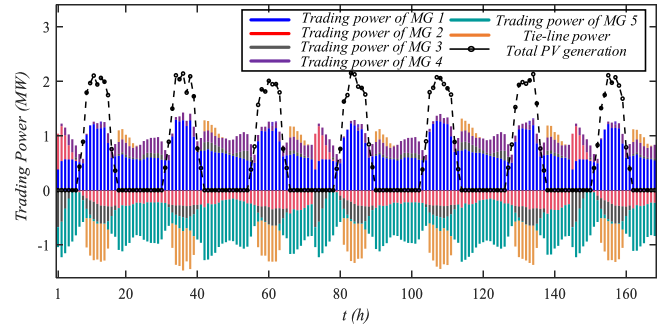

Fig. 7. Stacked histogram of long-term trading power via HRL and total PV.

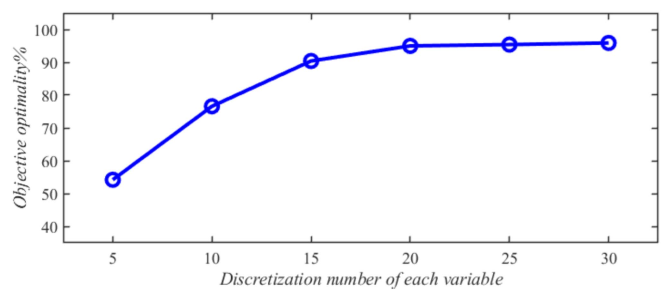

Fig. 8. The relationship between the discretization number of each variable and objective optimality.

Fig. 7 demonstartes the one-week (*168 h*) dispatch results of the proposed HRL. Further, Table I shows the standard deviation of trading power in a 1-week operation, where we notice that the trading power among MGs is regular and no sharp fluctuation is observed. Depending on the MG cost model, RES generation, and real-time load, the networked-MGs system sells surplus power to the bulk grid at noon and purchase shortage power in the late afternoon. All standard derivations of trading power are within a reasonable range, implying that HRL can ensure a stable operation of the system on a long-time scale.

Discretization leads to making a tradeoff between the optimality of the obtained solution and the size of the learning samples. The dispatch learning results with different discretization numbers of each variable are compared in Fig. 8. The Y-axis indicates the proportion of the operation cost to that of the optimal dispatch.

Fig. 8. The relationship between the discretization number of each variable and objective optimality.Based on the results, with the increase of the discretization sampling center, the objective optimality tends to be saturated and approaches 96%. When the number comes to 20, the increase in number has no obvious effect on the optimality improvement. Because online learning fails to acquire complete information, the inherent error will always exist.

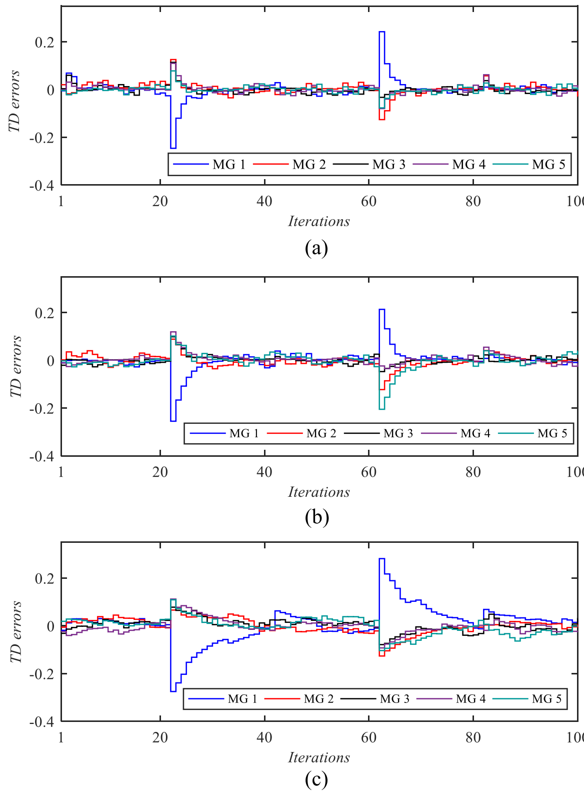

Fig. 9. TD errors in HRL-low-level iteration under system perturbation. a. $2\alpha_t = \beta_t = 1.4$. b. $2\alpha_t = \beta_t = 1.0$. c. $2\alpha_t = \beta_t = 0.6$.

C. Dynamic Results and Learning Rates Sensitive Analysis

Practical DOED should support various operation modes to cope with possible emergencies. We tested the TD error of low-level learning under system perturbation in Fig. 9 to highlight the dynamic adaptive ability of our online learning. 20 learning iterations are performed in an hour, and the gas turbine 5-2 exits and reconnects the networked MGs at the 20*th* and 60*th* iteration, respectively.

We also analyze the sensitivity of HRL to learning rates α_t and β_t. According to the results, the mismatch between estimated models and real ones introduces acceptable fluctuating TD errors. A sudden increase in TD errors is observed after the perturbations occur. Yet, approximation errors are reduced to a low level, which illustrates the algorithm has good self-adaption ability in dynamic learning with the increase of iteration. We can find that larger learning rates are beneficial to the convergence speed of learning iteration. However, as analyzed in Section *IV*, excessive speed rates eventually lead to system instability.

Appendix C also compares the online dynamic response between HRL based on *Greedy-Go* policy and fuzzy *Q*-learning based on the ε-*Greedy*.

D. Performance Comparison

We compare the proposed HRL algorithm with conventional fuzzy *Q*-learning and set discount factor γ=0.85. To fully demonstrate the superiority of the proposed online HRL, we compare it with general fuzzy *Q*-learning, and two other online algorithms: 1. online algorithm with current information (OACI) [31]. 2. online algorithm with dynamic optimization (OADO) [32].

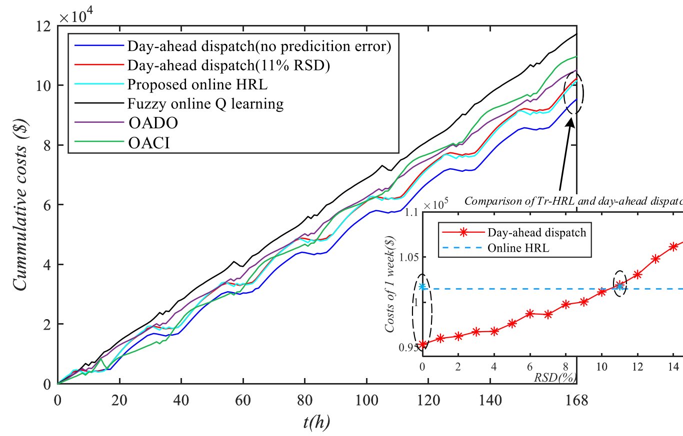

Fig. 10. Comparison of cumulative costs calculated by different algorithms.

This paper assumes that the above four distributed online algorithms can obtain real-time states without errors. A central day-ahead dispatch is used to provide a reference of the long-term optimal results. Considering the practical application of day-ahead dispatch, different random prediction errors are introduced in the comparison. Based on the correlation analysis of historical data, the correlation of prediction errors is taken into consideration. The performances of the day-ahead dispatch with different relative standard deviations (RSD) are provided in the following sub-figure to contrast the online HRL with day-ahead dispatch in non-ideal situations.

The four mentioned algorithms and our proposed HRL are tested in the one-week operation of networked MGs, which consists of 168 scheduling cycles (*168 h*). Cumulative costs optimized by the five algorithms are compared in Fig. 10. The cumulative costs decrease in several periods because the system injects surplus RES generation into the bulk grid.

Fig. 10 shows that the action of fuzzy *Q*-learning cannot obtain the optimal solution due to the discontinuous policy space of fuzzy *Q*-learning. Inevitable errors in day-ahead prediction introduce errors in dispatch results, which deteriorates its economic performance. However, other online dispatch methods can obtain real-time states without errors. OACI only uses current states to optimize real-time action, where no future rewards are considered. Although OADO takes future rewards into account, it cannot update the future rewards dynamically. This situation causes the operation point to approach feasible boundaries, which reduces the dispatchable margin in subsequent periods.

Our proposed HRL collects real-time data online and learns the current and future rewards by approximating the RBF. Moreover, the proposed HRL searches *Q* values in a continuous space without action discretization, which improves the learning efficiency. Although the proposed HRL is near-optimal not optimal, it outperforms other algorithms in the long-term operation economy when prediction errors are unacceptable, implying the high accuracy of HRL. We can further improve

TABLE I
THE STANDARD DEVIATION OF TRADING POWER IN A 1-WEEK OPERATION

Trading power	MG 1	MG 2	MG 3	MG 4	MG 5	Tie-line
Standard deviation (MW)	0.2187	0.1315	0.0941	0.0763	0.2314	0.1529

TABLE II
OPTIMIZATION PERFORMANCE OF DIFFERENT ECONOMIC DISPATCH ALGORITHMS IN 1 WEEK

Items		Manner	Cumulative costs ($)	BESS residual energy (%)	Samples in pre-learning	Abnormal conditions
Day-ahead dispatch	No prediction errors (Perfect reference)	Central	9.532×10^4	50.00	-	0
	11% RSD		10.211×10^4	53.13	-	6
Online dispatch	Proposed HRL	Distri.	10.147×10^4	49.29	3261	0
	Fuzzy Q-learning		11.721×10^4	41.35	6492	31
	OACI	Decentr.	10.967×10^4	28.27	-	17
	OADO		10.511×10^4	37.07	-	3

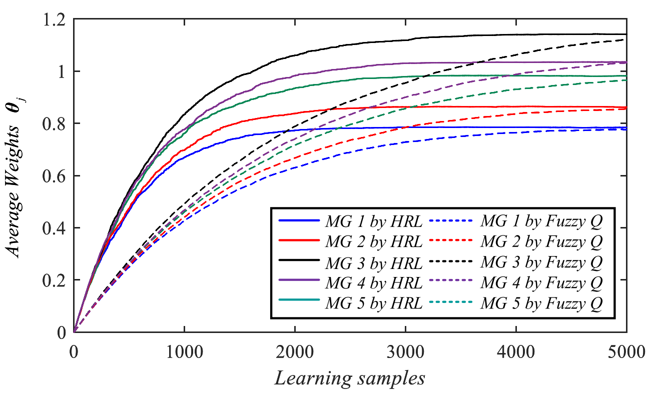

Fig. 11. Comparison of convergence processes in the offline pre-learning.

the solver accuracy by selecting a larger h at the expense of learning rates.

Table. II compares the optimization performance of different economic dispatch algorithms. Here, we define the abnormal condition as the cycles in which operation constraints are violated or reach constraint boundaries. Depending on the comparison, although day-ahead dispatch can obtain the optimal solution, in theory, the plan is unreliable in case of excessive error of day-ahead prediction. OACI is an online myopic algorithm. It applies to the short-term operation, whereas its long-term cumulative costs are high. Besides, BESS trends to discharge due to its short-sighted objective. OADO considers a fixed future reward rate for residual energy in BESS. It reduces the long-term operation costs, yet makes SOC reach its bounds in several periods.

General fuzzy Q-learning has poor online learning efficiency, it requires more samples to complete the pre-learning. In addition, its action space is discontinuous and the feasible region can only be weakly constrained by multiple penalty functions. The hierarchical architecture greatly reduced the demand size of offline learning samples.

Fig. 11 demonstrates the 5-MG convergence processes of HRL and general fuzzy Q-learning in offline learning. According

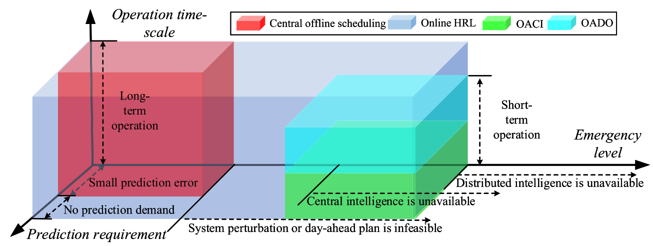

Fig. 12. Application scenario analysis of different scheduling modes.

to the results, HRL significantly outperforms fuzzy Q-learning in learning speed and efficiency. Based on the hierarchical architecture, HRL reduces the learning dimension and improves learning efficiency through principal component analysis. It has a strong modeling ability at the expense of accuracy to overcome the curse of dimensionality. However, a fuzzy mechanism makes its policy space discontinuous, and the *Epsilon-Greedy* strategy introduces random exploration. Both reduce the learning efficiency of the fuzzy Q-learning.

As is analyzed in Section III, the computational complexity of the proposed HRL is $O(2M^2 \cdot TN)$. Considering the entire learning space, $s_j = \{\rho, P_j^L - P_j^{RES}, SOC_j\}$ is the state set of traditional fuzzy Q-learning and the learning dimension is equal to 3. Similarly, there are at least two local policies and one global policy to learn $a_j = \{\sum P_j^I | j \in \mathcal{J}, P_j^B, \sum P_{j,i}^U | i \in \mathcal{I}_j\}$. Based on the analysis, the computational complexity of fuzzy Q-learning is $O[(2N+1)M^3 \cdot T]$. Therefore, the learning efficiency of HRL is much higher than that of fuzzy Q-learning, especially for cases including large amounts of MGs.

Therefore, the proposed HRL has the least cumulative costs and requires fewer samples in pre-learning. All operation constraints can always be satisfied in the best-action plane by embedding domain knowledge.

E. Application Analysis

The proposed distributed HRL dispatch learns the optimal strategy offline and modifies the existing strategy online at the same time, where only distributed intelligence is required. It can serve as an independent online dispatch with long-term objectives, or as a supplement to the central scheduling mode to enhance the robustness of the management systems. The application scenarios of different scheduling modes are analyzed in Fig. 12.

Generally, traditional central dispatch has the ability to obtain the optimal plans, which has good performance when central intelligence is available and prediction is accurate. However, predicition errors may lead to a sharp deterioration of scheduling performance. OACI and OADO support short-term operation via distributed intelligence. Based on RL, the proposed HRL learns the optimal strategy offline and modifies the existing strategy online. Furthermore, it not only adapts system perturbation but also obtains the long-term economy in an uncertain environment using distributed intelligence. Once the RL generates best-action policies, it can quickly provide the optimal operation policy online, unlike common distributed algorithms, (i.e. OACI and OADO), which need multiple iterations.

Therefore, our proposed HRL adapts to online dispatch and can serve as a standby dispatch for central dispatch in emergencies, such as system topology perturbation, excessive forecasting errors, and unavailable of central intelligence.

VI. CONCLUSION

In this paper, we developed a two-level HRL to dispatch networked MGs online in a distributed manner. Unlike day-ahead dispatch, online dispatch further streamlines the networked MGs using instant data with no prediction error. The simulation illustrates that our HRL outperforms fuzzy *Q*-learning and general online dispatch methods in supporting the long-term operation, including four aspects: *a*. hierarchical learning framework avoids the curse of dimensionality and requires fewer training samples. *b*. compared with OACI and OADO, TrHRL enables BESS to be dispatched from a long-term economic perspective. *c*. the proposed HRL can solve complex tasks with the local operation and global tie-line constraints without baseline violation by embedding domain knowledge. *d*. the proposed dispatch method achieves distributed online self-adaption and has highly improved performance in cost reduction and stable operation. Therefore, our proposed HRL has extensive adaptability and practicability.

Future work will consider power loss reduction in a distributed manner and extend the dispatch model to the system coupled with multiple energy.

APPENDIX A
DERIVATION AND LONG-TERM PERSPECTIVE PROOF

To illustrate the proposed HRL has a long-term perspective, we prove that the *Q*-function is derived from the expectation of long-term costs. For a sequence of action $\{a_j\}$, the long-term expectation costs can be described as

$$J_j\left(s_j, \{a_j\}\right) = \mathbb{E}\left[\sum_{t=1}^{\infty} \gamma^t r_j\left(s_j(t), a_j(t)\right) \middle| s_j(0) = s_j^0\right] \tag{35}$$

We define the optimal long-term rewards function.

$$V^*\left(s_j(t)\right) = \max_{a_j \in \mathcal{A}_t^j} J\left(s_j(t), \{a_j\}\right) \tag{36}$$

Assume $\mathcal{P}_j$ denotes the transition probability form $s_j(t)$ to $s_j(t+1)$. The function can be rewritten as an iterative equation.

$$V^*(s_j(t)) = \max_{a_j \in \mathcal{A}_t^j} \sum_{s_j(t+1)} \mathcal{P}_j\left[r_j\left(s_j(t), a_j(t)\right) + \gamma V^*\left(s_j(t+1)\right)\right] \tag{37}$$

Here, we prove that the optimal Q is equal to V^*, indicating that this Q updating process is equivalent to maximizing the expectation of long-term rewards.

$$Q^* = \sum_{s_j(t+1)} \mathcal{P}_j\left[r_j\left(s_j(t), a_j(t)\right) + \gamma V^*\left(s_j(t+1)\right)\right] \tag{38}$$

Q^* can be regarded as a fixed point of a contraction operator **H**, whose norms at any point are not magnified. q is defined as a decision mapping.

$$\begin{aligned} &Hq\left(s_j(t), a_j(t)\right) \\ &= \sum_{s_j(t+1)} \mathcal{P}_j\left[r_j(s_j(t), a_j(t)) + \gamma \max_{a_j \in \mathcal{A}_t^j} q(s_j(t+1), a_j(t+1))\right] \end{aligned} \tag{39}$$

Here, we prove that Q^* is a fixed point of a contraction operator **H** according to its definition.

$$\begin{aligned} \|Hq_1 - Hq_2\|_\infty &= \max_{s_j(t+1), a_j \in \mathcal{A}_t^j} \left| \sum_{s_j(t+1)} \mathcal{P}_j\left[r_j\left(s_j(t), a_j(t)\right) \right.\right. \\ &\quad + \gamma \max_{a_j \in \mathcal{A}_t^j} q_1\left(s_j(t+1), a_j(t+1)\right) - r_j\left(s_j(t), a_j(t)\right) \\ &\quad \left.\left. - \gamma \max_{a_j \in \mathcal{A}_t^j} q_2\left(s_j(t+1), a_j(t+1)\right)\right]\right| \\ &= \max_{s_j(t+1), a_j \in \mathcal{A}_t^j} \gamma \left| \sum_{s_j(t+1)} \mathcal{P}_j\left[\max_{a_j \in \mathcal{A}_t^j} q_1\left(s_j(t+1), a_j(t+1)\right) \right.\right. \\ &\quad \left.\left. - \max_{a_j \in \mathcal{A}_t^j} q_2\left(s_j(t+1), a_j(t+1)\right)\right]\right| \\ &\leqslant \max_{s_j(t+1), a_j \in \mathcal{A}_t^j} \gamma \sum_{s_j(t+1)} \mathcal{P}_j \left| \max_{a_j \in \mathcal{A}_t^j} q_1\left(s_j(t+1), a_j(t+1)\right) \right. \\ &\quad \left. - \max_{a_j \in \mathcal{A}_t^j} q_2\left(s_j(t+1), a_j(t+1)\right)\right| \\ &\leqslant \max_{s_j(t+1), a_j \in \mathcal{A}_t^j} \gamma \sum_{s_j(t+1)} \mathcal{P}_j \max_{a_j \in \mathcal{A}_t^j} \left|q_1\left(s_j(t+1), a_j(t+1)\right) \right. \\ &\quad \left. - q_2\left(s_j(t+1), a_j(t+1)\right)\right| \\ &= \max_{s_j(t+1), a_j \in \mathcal{A}_t^j} \gamma \sum_{s_j(t+1)} \mathcal{P}_j \|q_1 - q_2\|_\infty \\ &= \gamma \|q_1 - q_2\|_\infty \end{aligned} \tag{40}$$

Therefore, the *Q*-learning algorithm determines the optimal *Q*-function using sequential discrete samples of q. Taking the optimal *Q*-function as the reference, (16) updates *Q*-function by rate α_t.

In summary, the Q-function takes the expectation of long-term costs as the reference, and the proposed HRL has a long-term perspective.

APPENDIX B
CONDITIONS FOR DESIGNING TIME-VARYING RATES

In the *Greedy-Go* *Q*-learning with update equation (24), (25), the estimated *Q* will definitely converge to the unbiased estimation of optimal *Q*, if we select time-varying learning rates by the following principles:

1) $0 < \alpha_t < \beta_t < 1$.
2) $\sum_t \alpha_t = \sum_t \beta_t = \infty$.
3) $\sum_t \alpha_t^2 + \beta_t^2 < \infty$.

$\alpha_t > 0$ and $\beta_t > 0$ are necessary conditions indicating that θ_j and ω_j are updated along the direction of error reduction. Since the iteration of ω_j is nested in the iteration of θ_j. The equivalent learning rate of inner iteration should be the product of α_t and β_t. Thus, we generally have the condition: $\alpha_t < \beta_t$ to improve the learning efficiency of *Greedy-Go*.

To illustrate the necessity of the other conditions, we present an auxiliary theorem from stochastic approximation, which has been proven in ref. [33].

Theorem 1. The dynamic process defined as

$$\Delta_{t+1}(x) = (1-\chi_t)\Delta_{t+1}(x) + \chi_t O_t(x) \tag{41}$$

is bound to converge to zero, if the following conditions are all satisfied:

1) $0 \leqslant \chi_t \leqslant 1, \sum_t \chi_t = \infty, \sum_t {\chi_t}^2 < \infty$.
2) $\exists \kappa < 1, \|\mathbb{E}[O_t|\mathcal{F}_t]\|_W \leqslant \kappa \|\Delta_t\|_W$.
3) $\exists C > 0, \mathrm{var}[O_t|\mathcal{F}_t] \leqslant C(1+\|\Delta_t\|_W)$.

First, we can rewrite (16) as

$$Q_{t+1,j}(s_j,a_j) = (1-\alpha_t)Q_{t,j}(s_j,a_j) + \alpha_t \left[r_{t,j}(s_j,a_j) + \gamma \max_{a'_j} Q_{t,j}({s_j}',{a_j}') \right] \tag{42}$$

The process Δ_t is constructed:

$$\Delta_t = Q_{t,j}(s_j,a_j) - Q^*_{t,j}(s_j,a_j) \tag{43}$$

Further, we assume an auxiliary function O_t.

$$O_t(s_j,a_j) = r_{t,j}(s_j,a_j) + \gamma \max_{a'_j} Q_{t,j}({s_j}',{a_j}') - Q^*_{t,j}(s_j,a_j) \tag{44}$$

Both sides of (43) subtract $Q^*_{t,j}$, we can obtain

$$O_t(s_j,a_j) = r_{t,j}(s_j,a_j) + \gamma \max_{a'_j} Q_{t,j}({s_j}',{a_j}') - Q^*_{t,j}(s_j,a_j) \tag{45}$$

We define a contraction operator **H**. Considering policy transition probabilities, the expected O_t can be described as

$$\begin{aligned}\|\mathbb{E}\left[O_t\,|\mathcal{F}_t\right]\|_W &= \sum P\left(s_j,a_j\right) \Big[r_{t,j}(s_j,a_j) \\ &\quad + \gamma \max_{a'_j} Q_{t,j}({s_j}',{a_j}') - Q^*_{t,j}(s_j,a_j) \Big] \\ &= (HQ_{t,j})(s_j,a_j) - Q^*_{t,j}(s_j,a_j)\end{aligned} \tag{46}$$

Since $Q^*_{t,j} = HQ^*_{t,j}$, we can further obtain that

$$\|\mathbb{E}\left[O_t\,|\mathcal{F}_t\right]\|_W = (HQ_{t,j})(s_j,a_j) - HQ^*_{t,j} \tag{47}$$

According to the definition, any sup-sum of the operator **H** will be reduced.

$$\|\mathbb{E}\left[O_t\,|\mathcal{F}_t\right]\|_\infty \leqslant \kappa \left\|Q_{t,j} - Q^*_{t,j}\right\|_\infty \leqslant \kappa \|\Delta_t\|_\infty \tag{48}$$

Here, we can obtain the variance of $\|\mathbb{E}[O_t|\mathcal{F}_t]\|$.

$$\begin{aligned}\mathrm{var}\,\|\mathbb{E}\left[O_t\,|\mathcal{F}_t\right]\| &= \mathbb{E}\left[\Big(r_{t,j}(s_j,a_j) + \gamma \max_{a'_j} Q_{t,j}({s_j}',{a_j}') \right. \\ &\quad \left. - Q^*_{t,j}(s_j,a_j) - (HQ_{t,j})(s_j,a_j) + Q^*_{t,j} \Big)^2 \right] \\ &= \mathbb{E}\Big[\Big(r_{t,j}(s_j,a_j) + \gamma \max_{a'_j} Q_{t,j}({s_j}',{a_j}') \\ &\quad - (HQ_{t,j})(s_j,a_j) \Big)^2 \Big] \\ &= \mathrm{var}\left[r_{t,j}(s_j,a_j) + \gamma \max_{a'_j} Q_{t,j}({s_j}',{a_j}')\,|\mathcal{F}_t \right]\end{aligned} \tag{49}$$

Obviously, the immediate revenue is bounded in our model. Thus, we can clearly verify that

$$\mathrm{var}\left[O_t\,|\mathcal{F}_t\right] \leqslant C(1+\|\Delta_t\|_W) \tag{50}$$

Based on (48), (50), and **Theorem 1**, the Q function and its weights of basic functions are bound to converge to zero if $\alpha_t \leqslant 1, \sum_t \alpha_t = \infty, \sum_t {\alpha_t}^2 < \infty$.

Similarly, we can construct the process Δ'_t.

$$\Delta'_t = \omega_j(t) - \omega^*_j(t) \tag{51}$$

$$\begin{aligned}\Delta'_t =& (1-\beta_t)\Delta'_t \\ &+ \beta_t \left[\varphi_j(t)\varepsilon_j(t) + (\varphi_j(t)-1)\,\varphi_j(t)^T\omega_j(t) - \omega^*_j(t) \right]\end{aligned} \tag{52}$$

This time, we assume another auxiliary function

$$O'_t = \varphi_j(t)\varepsilon_j(t) - (1+\varphi_j(t))\,\varphi_j(t)^T\omega_j(t) + \omega^*_j(t) \tag{53}$$

The expected value of O'_t can be described as

$$\begin{aligned}&\|\mathbb{E}\left[O_t\,|\mathcal{F}'_t\right]\|_W \\ &= \sum P\left(s_j,a_j\right) \\ &\quad \times \left[\varphi_j(t)\varepsilon_j(t) + (\varphi_j(t)-1)\,\varphi_j(t)^T\omega_j(t) - \omega^*_j(t) \right] \\ &= (H\omega_j)(t) - \omega^*_j(t) = (H\omega_j)(t) - H\omega^*_j(t)\end{aligned} \tag{54}$$

H is another contraction operator. Therefore, according to the definition, we can find a parameter κ' satisfying

$$\|\mathbb{E}\left[{O'}_t\,|\mathcal{F}_t\right]\|_\infty \leqslant \kappa' \|\omega_j - \omega^*_t\|_\infty \leqslant \kappa' \|{\Delta'}_t\|_\infty \tag{55}$$

Similarly, we can verify

$$\begin{aligned}&\mathrm{var}\,\|\mathbb{E}\left[{O'}_t\,|\mathcal{F}_t\right]\| \\ &= \mathrm{var}\left[\varphi_j(t)\varepsilon_j(t) + (\varphi_j(t)-1)\,\varphi_j(t)^T\omega_j(t)\,|\mathcal{F}_t \right]\end{aligned} \tag{56}$$

Since φ_j and ε_j are bounded, $\varphi_j(t)\varepsilon_j(t)$ is bounded, and we can find a constant C' satisfying

$$\mathrm{var}\left[{O'}_t\,|\mathcal{F}_t\right] \leqslant C'(1+\|{\Delta'}_t\|_W) \tag{57}$$

Therefore, the two processes are bound to converge to zero if the first condition in **Theorem 1** is satisfied. To sum up, the three given conditions for selecting time-varying rates can guarantee the convergence of the algorithm.

Appendix C
Supplemented Analysis and Comparison

A. Correlation Analysis of State Variables

This part provides a hierarchical foundation for deciding the input variables of each level. Here, we analyze the correlation among the environment states, bilevel learning inputs, and outputs from two aspects: qualitative and quantitative bases.

Qualitative analysis:

According to the high-level design in Fig. 2, the available state variables are electricity price, total load and, the SOC variables

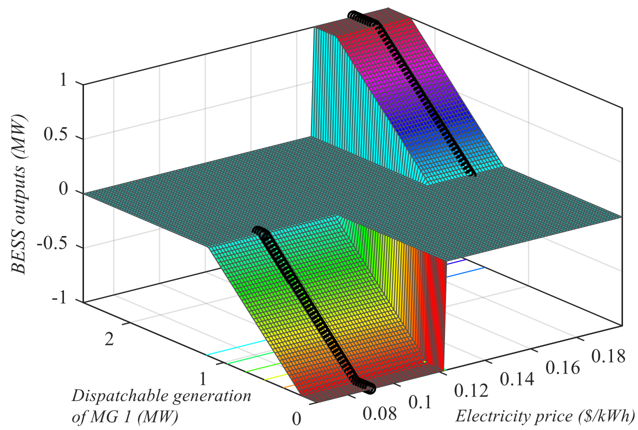

Fig. 13. Correlation analysis of electricity price and dispatchable generation to BESS outputs.

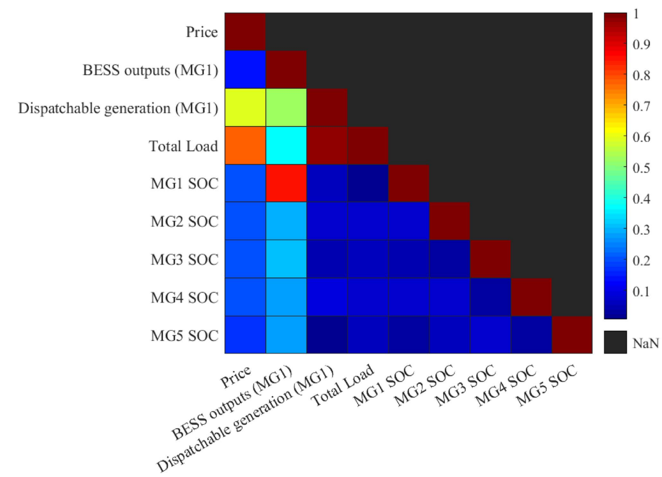

Fig. 14. Color map of the correlation coefficient matrix.

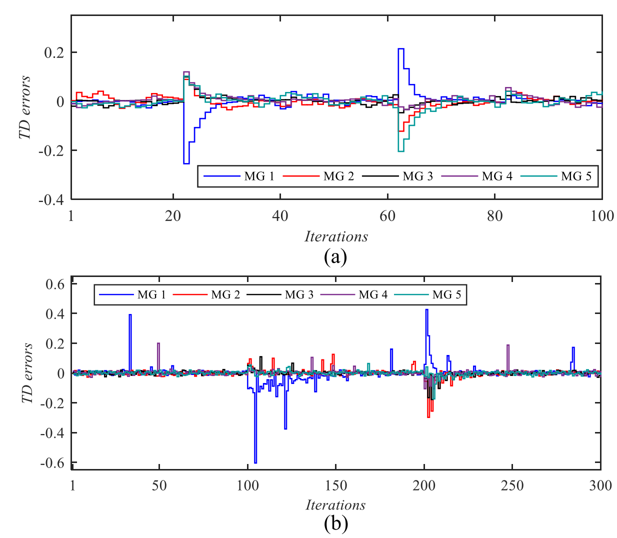

Fig. 15. TD errors in fuzzy Q-learning iterations. (a) HRL based on Greedy-GO strategy ($2\alpha_t = \beta_t = 1.0$). (b) Fuzzy Q-learning based on Epsilon-Greedy strategy ($\alpha_t = 0.5$).

of each MG. The overall tie-line power will be optimized by high-level learning based on the price signal because electricity price is already considered in the high-level Lagrange penalty item $\Upsilon_j(s_j, a_j)$. This learning space can be simplified by regarding the price factor as an input state. The high-level learning provides the best-action policy for a certain price. Therefore, the learning space is a 2-D space with states-SOC and total load.

As to low-level learning, since the SOC determines the margin of BESS charging/discharging, we default that the equivalent cost of SOC is strongly related to BESS outputs according to (5). We analyze the correlation between electricity price and dispatchable generation to BESS outputs, as depicted in Fig. 13. Based on the results, electricity price is relatively independent of BESS outputs if domain knowledge is applied. Generation allocation is determined only by internal operation characteristics once the total dispatchable generation is given. Once the BESS outputs are given, we can derive the local generation of dispatchable units, and local agents can optimize the power sharing according to the equal increment rate principle. Therefore, the electricity price factor can be removed from the state set of low-level if a proper knowledge set is established.

Quantitative analysis:

Based on the results of optimal dispatch, the correlation matrix of the variable set is calculated here. The variable set includes 9 variables: price ρ, BESS outputs of MG 1 P_1^B, dispatchable generation of MG 1 $P_1^B + \sum P_{1,i}^U$, the total equivalent load of MG 1 $P_1^L - P_1^{RES}$, and state of charge of 5 MGs $SOC_j, j \in [1, 5]$. Fig. 14 illustrates the color map of the correlation coefficient matrix.

According to the matrix, BESS outputs and dispatchable generation are strongly correlated with total load and local state of charge, respectively. The total load is also correlated with price. However, they are all known variables. This provides the basis for the bilevel learning design.

B. Dynamic Response Comparison of Learning

This part compares the dynamic response performance between *Greedy-Go*-based HRL and ε-*Greedy* based fuzzy Q-learning. The TD errors of the following two strategies with the same α_t are demonstrated in Fig. 15.

In Fig. 15 a, 20 learning iterations are performed in an hour, and the gas turbine 5-2 exits and reconnects the networked MGs at the 20th and 60th iteration, respectively. While in Fig. 15 b, 100 learning iterations are performed in an hour, and the gas turbine 5-2 exits and reconnects the networked MGs at the 100*th* and 200*th* iteration, respectively. Based on the results, it can be found that the proposed HRL based on the *Greedy-Go* strategy outperforms the fuzzy Q-learning in dynamic response speed. Large exploration probability introduces spurts in several periods, resulting in irrational actions. These irrational actions are unacceptable for a power system, which has high requirements for scheduling security. In addition, not only the action policy but also the TD errors are perturbated after a plug-&-play operation because the action space of fuzzy Q-learning is discontinuous. In addition, the discrete decision policy slows the learning rate at the same time. Therefore, the average TD errors of HRL are much smaller than that of fuzzy Q-learning.

APPENDIX D
CONFIGURATION AND OPERATION PARAMETERS

Parameters	MG 1	MG 2	MG 3	MG 4	MG 5
Initial SOC (%)	50	50	50	50	50
BESS capacity (MWh)	5	6	4	5	5.5
$\overline{P}_j^{B,dis}$, $\underline{P}_j^{B,ch}$ (MW)	1, -1	1, -1	1, -1	1, -1	1, -1
$\underline{SOC}_j, \overline{SOC}_j$	0.1, 0.9	0.1, 0.9	0.1, 0.9	0.1, 0.9	0.1, 0.9
σ_j^{ch} ($/MWh)	[8.96,11.94	[8.96,11.94	[8.96,11.94	[8.96,11.94	[8.96,11.94
SOC: [0,0.45,0.6,0.9]	16.42,20.90]	16.42,20.90]	16.42,20.90]	16.42,20.90]	16.42,20.90]
σ_j^{dis} ($/MWh)	[29.85,14.93	[29.85,14.93	[29.85,14.93	[29.85,14.93	[29.85,14.93
SOC: [0,0.45,0.6,0.9]	11.94, 10.45]	11.94, 10.45]	11.94, 10.45]	11.94, 10.45]	11.94, 10.45]
$\eta_j^{ch}, \eta_j^{dis}$	0.95, 0.98	0.95, 0.98	0.95, 0.98	0.95, 0.98	0.95, 0.98
$\underline{P}_{j,i}^{U}, \overline{P}_{j,i}^{U}$ (MW)	0.3, 2.0	0.4, 2.5	0.3, 2.0	0.2, 1.5	0.4, 2.5
$a_{n,i}$ ($/MW^2)	5.443	7.929	9.000	8.500	9.581
$b_{n,i}$ ($/MW)	82.857	80.000	64.286	71.429	67.143

REFERENCES

[1] T. Lu, P. Sherman, X. Chen, S. Chen, X. Lu, and M. B. McElroy, “India’s potential for integrating solar and on- and offshore wind power into its energy system,” *Nature Commun.*, vol. 11, no. 1, pp. 1–10, 2020.

[2] T. Lv and Q. Ai, “Interactive energy management of networked microgrids-based active distribution system considering large-scale integration of renewable energy resources,” *Appl. Energy*, vol. 163, pp. 408–422, 2016.

[3] T. Lu, Z. Wang, J. Wang, Q. Ai, and C. Wang, “A data-driven stackelberg market strategy for demand response-enabled distribution systems,” *IEEE Trans. Smart Grid*, vol. 10, no. 3, pp. 2345–2357, May 2019.

[4] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Massachusetts: The MIT Press, 2016.

[5] T. S. Borkar and L. J. Karam, “Deepcorrect: Correcting DNN models against image distortions,” *IEEE Trans. Image Process.*, vol. 28, no. 12, pp. 6022–6034, Dec. 2019.

[6] R. Hao, T. Lu, and Q. Ai, “Distributed online learning and dynamic robust standby dispatch for networked microgrids,” *Appl. Energy*, vol. 274, 2020, Art. no. 115256.

[7] Q. Wei, D. Liu, and G. Shi, “A novel dual iterative Q-learning method for optimal battery management in smart residential environments,” *IEEE Trans. Ind. Electron.*, vol. 62, no. 4, pp. 2509–2518, Apr. 2015.

[8] H. Xu, H. Sun, D. Nikovski, S. Kitamura, K. Mori, and H. Hashimoto, “Deep reinforcement learning for joint bidding and pricing of load serving entity,” *IEEE Trans. Smart Grid*, vol. 10, no. 6, pp. 6366–6375, Nov. 2019.

[9] T. Lu, X. Chen, M. B. McElroy, C. P. Nielsen, Q. Wu, and Q. Ai, “A reinforcement learning-based decision system for electricity pricing plan selection by smart grid end users,” *IEEE Trans. Smart Grid*, vol. 12, no. 3, pp. 2176–2187, May 2021.

[10] Y. Chen, S. Huang, F. Liu, Z. Wang, and X. Sun, “Evaluation of reinforcement learning-based false data injection attack to automatic voltage control,” *IEEE Trans. Smart Grid*, vol. 10, no. 2, pp. 2158–2169, Mar. 2019.

[11] J. R. Vázquez-Canteli and Z. Nagy, “Reinforcement learning for demand response: A review of algorithms and modeling techniques,” *Appl. Energy*, vol. 305, pp. 1072–1089, 2019.

[12] R. Lu, S. H. Hong, and X. Zhang, “A dynamic pricing demand response algorithm for smart grid: Reinforcement learning approach,” *Appl. Energy*, vol. 220, pp. 220–230, 2018.

[13] Y. Du and F. Li, “Intelligent multi-microgrid energy management based on deep neural network and model-free reinforcement learning,” *IEEE Trans. Smart Grid*, vol. 11, no. 2, pp. 1066–1076, Mar. 2019.

[14] X. Zhang, T. Yu, Z. Zhang, and J. Tang, “Multi-agent bargaining learning for distributed energy hub economic dispatch,” *IEEE Access*, vol. 6, pp. 39564–39573, 2018.

[15] E. Foruzan, L. Soh, and S. Asgarpoor, “Reinforcement learning approach for optimal distributed energy management in a microgrid,” *IEEE Trans. Power Syst.*, vol. 33, no. 5, pp. 5749–5758, Sep. 2018.

[16] X. Lu, X. Xiao, L. Xiao, C. Dai, M. Peng, and H. V. Poor, “Reinforcement learning-based microgrid energy trading with a reduced power plant schedule,” *IEEE Internet Things J.*, vol. 6, no. 6, pp. 10 728–10 737, Dec. 2019.

[17] P. Kofinas, A. I. Dounis, and G. A. Vouros, “Fuzzy q-learning for multi-agent decentralized energy management in microgrids,” *Appl. Energy*, vol. 219, pp. 53–67, 2018.

[18] Z. Zhang, D. Zhao, D. W. J. Gao, and Y. Dai, “Fmrq-a multiagent reinforcement learning algorithm for fully cooperative tasks,” *IEEE Trans. Cybern.*, vol. 47, no. 6, pp. 1367–1379, Jun. 2017.

[19] R. S. Sutton, H. R. Maei, D. Precup, S. Bhatnagar, C. S. D. Silver, and E. Wiewiora, “Fast gradient-descent methods for temporal difference learning with linear function approximation,” in *Proc. 26th Annu. Int. Conf. Mach. Learn.*, 2009, pp. 993–1000.

[20] R. Zafar, J. Ravishankar, J. E. Fletc, and H. R. Pota, “Optimal dispatch of battery energy storage system using convex relaxations in unbalanced distribution grids,” *IEEE Trans. Indust. Inform.*, vol. 16, no. 1, pp. 97-108, Jan. 2020.

[21] N. Lei, K. Su, L. Cui, S. T. Yau, and D. X. Gu, “A geometric view of optimal transporta-tion and generative model,” *arXiv:1710.05488*, 2017.

[22] J. Nocedal and S. Wright, *Numerical Optimization*. New York, NY, USA: Springer Science and Business Media, 2006.

[23] H. Xing, Y. Mou, M. Fu, and Z. Lin, “Distributed bisection method for economic power dispatch in smart grid,” *IEEE Trans. Power Syst.*, vol. 30, no. 6, pp. 3024–3035, Nov. 2015.

[24] O. Nachum, S. S. Gu, H. Lee, and S. Levine, “Data-efficient hierarchical reinforcement learning,” in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, pp. 3303–3313.

[25] T. D. Kulkarni, K. Narasimhan, A. Saeedi, and J. Tenenbaum, “Hierarchical deep reinforcement learning: Integrating temporal abstraction and intrinsic motivation,” in *Proc. Adv. Neural Inf. Process. Syst.*, pp. 3675–3683, 2016.

[26] J. Lago, F. D. Ridder, P. Vrancx, and B. D. Schutter, “Forecasting day-ahead electricity prices in Europe: The importance of considering market integration,” *Appl. Energy*, vol. 211, pp. 890–903, 2018.

[27] L. Busoniu, R. Babuska, B. D. Schutter, and D. Ernst, *Reinforcement Learning and Dynamic Programming Using Function Approximators*. Boca Raton, FL, USA: CRC Press, 2010.

[28] H. R. Maei, C. Szepesvari, S. Bhatnagar, and R. S. Sutton, “Toward off-policy learning control with function approximation,” in *Proc. 27th Int. Conf. Mach. Learn.*, 2010, pp. 1–8.

[29] W. Liu, P. Zhuang, J. P. H. Liang, and Z. Huang, “Distributed economic dispatch in microgrids based on cooperative reinforcement learning,” *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 29, no. 6, pp. 2192–2203, Jun. 2018.

[30] S. V. Macua, J. Chen, S. Zazo, and A. H. Sayed, “Distributed policy evaluation under multiple behavior strategies,” *IEEE Trans. Autom. Control*, vol. 60, no. 5, pp. 1260–1274, May 2015.

[31] J. Yan *et al.*, “Real-time energy management for a smart-community microgrid with battery swapping and renewables,” *Appl. Energy*, vol. 238, pp. 180–194, 2019.

[32] S. Salinas, M. Li, P. Li, and Y. Fu, “Dynamic energy management for the smart grid with distributed energy resources,” *IEEE Trans. Smart Grid*, vol. 4, no. 4, pp. 2139–2150, Dec. 2013.

[33] T. Jaakkola, M. I. Jordan, and S. P. Singh, “Convergence of stochastic iterative dynamic programming algorithms,” in *Proc. Adv. Neural Inf. Process. Syst.*, 1994, pp. 703–710.